

Tarea M20-CD – Análisis de Regresión Ridge y Lasso

Profesión: Científico de Datos v2

Objetivo

El objetivo de esta práctica es construir y comparar modelos de regresión múltiple, Ridge y Lasso para pronosticar las emisiones de CO₂ a partir de variables técnicas de consumo de combustible, evaluando su desempeño mediante métricas estadísticas y seleccionando el modelo con mejor capacidad predictiva.

Descripción general

Se utiliza una base de datos real relacionada con el consumo de combustible de vehículos y sus emisiones de CO₂. El análisis incluye la limpieza de datos, el entrenamiento de distintos modelos de regresión y la comparación de resultados con base en métricas de bondad de ajuste.

Autor

RobertsScience – Consultoría en Ciencia de Datos

```
In [33]: # =====  
# Importación de Librerías  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression, Ridge, Lasso  
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error  
  
pd.set_option("display.max_columns", None)  
plt.style.use("default")
```

```
In [34]: # =====  
# Carga de datos  
# =====  
  
ruta_datos = "../data/FuelConsumptionCo2.xlsx"  
df = pd.read_excel(ruta_datos)  
  
df.head()
```

Out[34]:

	MODELYEAR	MAKE	MODEL	VEHICLECLASS	ENGINE SIZE	CYLINDERS	TRANSMISSION
0	2022	Acura	ILX	Compact	2.4	4	A
1	2022	Acura	MDX SH-AWD	SUV: Small	3.5	6	A
2	2022	Acura	RDX SH-AWD	SUV: Small	2.0	4	A
3	2022	Acura	RDX SH-AWD A-SPEC	SUV: Small	2.0	4	A
4	2022	Acura	TLX SH-AWD	Compact	2.0	4	A

In [35]:

```
# =====
# Exploración inicial
# =====

df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 945 entries, 0 to 944
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   MODELYEAR                            945 non-null    int64
1   MAKE                                945 non-null    str
2   MODEL                                945 non-null    str
3   VEHICLECLASS                         945 non-null    str
4   ENGINE SIZE                          945 non-null    float64
5   CYLINDERS                            945 non-null    int64
6   TRANSMISSION                        945 non-null    str
7   FUELTYPE                             945 non-null    str
8   FUELCONSUMPTION_CITY                 945 non-null    float64
9   FUELCONSUMPTION_HWY                 945 non-null    float64
10  FUELCONSUMPTION_COMB                 945 non-null    float64
11  FUELCONSUMPTION_COMB_MPG            945 non-null    int64
12  CO2EMISSIONS                        945 non-null    int64
dtypes: float64(4), int64(4), str(5)
memory usage: 96.1 KB
```

In [36]:

```
# =====
# Estadísticos descriptivos
# =====

df.describe()
```

Out[36]:

	MODELYEAR	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_CITY	FUELCONSUMI
count	945.0	945.000000	945.000000	945.000000	
mean	2022.0	3.201058	5.670899	12.515767	
std	0.0	1.374256	1.932837	3.452369	
min	2022.0	1.200000	3.000000	4.000000	
25%	2022.0	2.000000	4.000000	10.200000	
50%	2022.0	3.000000	6.000000	12.200000	
75%	2022.0	3.800000	6.000000	14.700000	
max	2022.0	8.000000	16.000000	30.300000	

In [37]:

```
# =====
# Limpieza de datos
# =====

df_limpio = df.dropna().copy()
df_limpio.isnull().sum()
```

Out[37]:

```
MODELYEAR      0
MAKE           0
MODEL          0
VEHICLECLASS   0
ENGINESIZE     0
CYLINDERS      0
TRANSMISSION   0
FUELTYPE       0
FUELCONSUMPTION_CITY  0
FUELCONSUMPTION_HWY  0
FUELCONSUMPTION_COMB  0
FUELCONSUMPTION_COMB_MPG  0
CO2EMISSIONS    0
dtype: int64
```

In [38]:

```
# =====
# Definición de variables X e y
# =====

# Variable objetivo
y = df_limpio["CO2EMISSIONS"]

# Variables predictoras
X = df_limpio.drop(columns=["CO2EMISSIONS"])

X.head(), y.head()
```

```
Out[38]: (  MODELYEAR  MAKE                MODEL VEHICLECLASS  ENGINE SIZE  CYLINDERS  \
0         2022  Acura                ILX      Compact        2.4         4
1         2022  Acura      MDX SH-AWD  SUV: Small        3.5         6
2         2022  Acura      RDX SH-AWD  SUV: Small        2.0         4
3         2022  Acura  RDX SH-AWD A-SPEC  SUV: Small        2.0         4
4         2022  Acura      TLX SH-AWD    Compact        2.0         4

  TRANSMISSION  FUELTYPE  FUELCONSUMPTION_CITY  FUELCONSUMPTION_HWY  \
0          AM8         Z              9.9          7.0
1         AS10         Z             12.6          9.4
2         AS10         Z             11.0          8.6
3         AS10         Z             11.3          9.1
4         AS10         Z             11.2          8.0

  FUELCONSUMPTION_COMB  FUELCONSUMPTION_COMB_MPG
0              8.6          33
1             11.2          25
2              9.9          29
3             10.3          27
4              9.8          29 ,
0      200
1      263
2      232
3      242
4      230
Name: CO2EMISSIONS, dtype: int64)
```

```
In [39]: # =====
# División de datos
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape
```

```
Out[39]: ((756, 12), (189, 12))
```

Preparación de los datos

Con el objetivo de garantizar la correcta aplicación de los modelos de regresión, se realizó una depuración previa de la base de datos. En esta etapa se eliminaron las variables de tipo categórico, conservando únicamente aquellas de naturaleza numérica, ya que los modelos de regresión lineal requieren variables cuantitativas como predictores.

Adicionalmente, se verificó la ausencia de valores nulos en las variables seleccionadas para asegurar la consistencia del análisis.

```
In [40]: # =====
# Selección de variables numéricas
# =====

# Conservamos únicamente variables numéricas
df_modelo = df.select_dtypes(include=[np.number])
```

```
# Verificación de valores nulos
df_modelo.isnull().sum()
```

```
Out[40]: MODELYEAR          0
          ENGINE SIZE      0
          CYLINDERS        0
          FUELCONSUMPTION_CITY  0
          FUELCONSUMPTION_HWY  0
          FUELCONSUMPTION_COMB  0
          FUELCONSUMPTION_COMB_MPG  0
          CO2EMISSIONS      0
          dtype: int64
```

```
In [41]: # =====
# Definición de variables dependiente e independientes
# =====

y = df_modelo["CO2EMISSIONS"]
X = df_modelo.drop(columns=["CO2EMISSIONS"])

X.head(), y.head()
```

```
Out[41]: (  MODELYEAR  ENGINE SIZE  CYLINDERS  FUELCONSUMPTION_CITY  \
0         2022         2.4         4          9.9
1         2022         3.5         6         12.6
2         2022         2.0         4         11.0
3         2022         2.0         4         11.3
4         2022         2.0         4         11.2

          FUELCONSUMPTION_HWY  FUELCONSUMPTION_COMB  FUELCONSUMPTION_COMB_MPG
0              7.0              8.6              33
1              9.4             11.2              25
2              8.6              9.9              29
3              9.1             10.3              27
4              8.0              9.8              29 ,
0         200
1         263
2         232
3         242
4         230
          Name: CO2EMISSIONS, dtype: int64)
```

```
In [42]: # =====
# División en entrenamiento y prueba
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape
```

```
Out[42]: ((756, 7), (189, 7))
```

Modelo de Regresión Lineal Múltiple

Como modelo base, se implementó una regresión lineal múltiple con el objetivo de establecer un punto de comparación frente a los modelos regularizados. Este modelo permite analizar la relación lineal entre las variables explicativas y la variable objetivo, asumiendo independencia entre predictores y ausencia de penalización en los coeficientes.

Los resultados obtenidos servirán como referencia para evaluar el impacto de la regularización aplicada en los modelos Ridge y Lasso.

```
In [43]: # =====  
# Regresión Lineal Múltiple (modelo base)  
# =====  
  
modelo_lr = LinearRegression()  
modelo_lr.fit(X_train, y_train)  
  
y_pred_lr = modelo_lr.predict(X_test)  
  
r2_lr = r2_score(y_test, y_pred_lr)  
mae_lr = mean_absolute_error(y_test, y_pred_lr)  
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))  
  
r2_lr, mae_lr, rmse_lr
```

```
Out[43]: (0.9004924315633445, 7.33914492064647, np.float64(19.205917329752))
```

Modelo de Regresión Ridge

Se implementó un modelo de regresión Ridge con el objetivo de reducir la varianza del modelo mediante regularización L2. Para su estimación se evaluaron distintos valores del parámetro α , seleccionando aquel que maximiza el desempeño predictivo sobre el conjunto de prueba.

```
In [44]: # =====  
# Selección del parámetro alpha para Ridge  
# =====  
  
alphas = np.logspace(-3, 3, 50)  
r2_scores_ridge = []  
  
for a in alphas:  
    ridge = Ridge(alpha=a)  
    ridge.fit(X_train, y_train)  
    preds = ridge.predict(X_test)  
    r2_scores_ridge.append(r2_score(y_test, preds))  
  
alpha_ridge_opt = alphas[np.argmax(r2_scores_ridge)]  
alpha_ridge_opt
```

```
Out[44]: np.float64(1000.0)
```

```
In [45]: # =====  
# Modelo Ridge final  
# =====
```

```

modelo_ridge = Ridge(alpha=alpha_ridge_opt)
modelo_ridge.fit(X_train, y_train)

y_pred_ridge = modelo_ridge.predict(X_test)

r2_ridge = r2_score(y_test, y_pred_ridge)
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))

r2_ridge, mae_ridge, rmse_ridge

```

Out[45]: (0.9170316019722166, 8.401420160606623, np.float64(17.537325006797673))

Modelo de Regresión Lasso

Se aplicó un modelo de regresión Lasso con regularización L1, el cual permite realizar selección automática de variables al penalizar los coeficientes menos relevantes. Se evaluaron distintos valores del parámetro α para identificar el modelo con mejor desempeño predictivo.

```

In [46]: # =====
# Selección del parámetro alpha para Lasso
# =====

alphas = np.logspace(-3, 1, 50)
r2_scores_lasso = []

for a in alphas:
    lasso = Lasso(alpha=a, max_iter=10000)
    lasso.fit(X_train, y_train)
    preds = lasso.predict(X_test)
    r2_scores_lasso.append(r2_score(y_test, preds))

alpha_lasso_opt = alphas[np.argmax(r2_scores_lasso)]
alpha_lasso_opt

```

```

c:\Users\ROBERTSCIENCE\Documents\Tarea M20-CD - robertscience\.venv\Lib\site-pac
kages\sklearn\linear_model\_coordinate_descent.py:716: ConvergenceWarning: Object
ive did not converge. You might want to increase the number of iterations, check
the scale of the features or consider increasing regularisation. Duality gap: 1.6
98e+04, tolerance: 3.223e+02
    model = cd_fast.enet_coordinate_descent(
c:\Users\ROBERTSCIENCE\Documents\Tarea M20-CD - robertscience\.venv\Lib\site-pac
kages\sklearn\linear_model\_coordinate_descent.py:716: ConvergenceWarning: Object
ive did not converge. You might want to increase the number of iterations, check
the scale of the features or consider increasing regularisation. Duality gap: 8.5
77e+03, tolerance: 3.223e+02
    model = cd_fast.enet_coordinate_descent(
c:\Users\ROBERTSCIENCE\Documents\Tarea M20-CD - robertscience\.venv\Lib\site-pac
kages\sklearn\linear_model\_coordinate_descent.py:716: ConvergenceWarning: Object
ive did not converge. You might want to increase the number of iterations, check
the scale of the features or consider increasing regularisation. Duality gap: 1.1
49e+03, tolerance: 3.223e+02
    model = cd_fast.enet_coordinate_descent(

```

Out[46]: np.float64(10.0)

```
In [47]: # =====
# Modelo Lasso final
# =====

modelo_lasso = Lasso(alpha=alpha_lasso_opt, max_iter=10000)
modelo_lasso.fit(X_train, y_train)

y_pred_lasso = modelo_lasso.predict(X_test)

r2_lasso = r2_score(y_test, y_pred_lasso)
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))

r2_lasso, mae_lasso, rmse_lasso
```

```
Out[47]: (0.9077436800391347, 10.380151689677277, np.float64(18.492901720014547))
```

Comparación de modelos

A continuación, se presenta una comparación entre los modelos de Regresión Lineal Múltiple, Ridge y Lasso utilizando métricas de desempeño sobre el conjunto de prueba.

Pronóstico de una observación individual

Con el objetivo de validar el funcionamiento del modelo, se realizó el pronóstico de la primera observación del conjunto de prueba utilizando directamente los coeficientes del modelo y se comparó con el resultado obtenido mediante la función `predict()` de scikit-learn.

```
In [48]: # =====
# Pronóstico manual vs predict()
# =====

# Primera observación del conjunto de prueba
x_primera = X_test.iloc[0]
y_real = y_test.iloc[0]

# Pronóstico manual usando coeficientes
y_manual = modelo_lr.intercept_ + np.dot(modelo_lr.coef_, x_primera)

# Pronóstico con predict
y_predict = modelo_lr.predict(X_test.iloc[[0]])[0]

y_real, y_manual, y_predict
```

```
Out[48]: (np.int64(247), np.float64(242.92694037393747), np.float64(242.92694037393747))
```

Los resultados obtenidos mediante el cálculo manual y la función `predict()` coinciden, lo cual confirma la correcta implementación del modelo de regresión lineal múltiple.

```
In [49]: # =====
# Tabla comparativa de modelos
```



```
# =====

resultados = pd.DataFrame({
    "Modelo": ["Regresión Lineal", "Ridge", "Lasso"],
    "R²": [r2_lr, r2_ridge, r2_lasso],
    "MAE": [mae_lr, mae_ridge, mae_lasso],
    "RMSE": [rmse_lr, rmse_ridge, rmse_lasso]
})

resultados
```

Out[49]:

	Modelo	R ²	MAE	RMSE
0	Regresión Lineal	0.900492	7.339145	19.205917
1	Ridge	0.917032	8.401420	17.537325
2	Lasso	0.907744	10.380152	18.492902

Conclusiones

A partir de los resultados obtenidos, se observa que el modelo de regresión que presenta el mejor desempeño es el modelo **Ridge**, ya que obtuvo el mayor valor de R^2 y los menores errores MAE y RMSE en comparación con los modelos de Regresión Lineal Múltiple y Lasso.

El modelo Ridge logra un balance adecuado entre ajuste y regularización, reduciendo la varianza sin eliminar completamente variables relevantes, mientras que el modelo Lasso penaliza con mayor fuerza los coeficientes, lo cual puede afectar su capacidad predictiva en este caso.

Por lo anterior, se concluye que el modelo seleccionado es el más adecuado para realizar pronósticos de emisiones de CO₂ con base en las variables disponibles.